Chuyên đề: Số học cơ bản (Phần 1)

Các thuật toán nền tảng trong Competitive Programming

Chuyên Đề Thuật Toán

Nội dung chính

1. Ước số và Bội số
2. Số nguyên tố
3. Ước chung lớn nhất (GCD) & Bội chung nhỏ nhất (LCM)
4. Phân tích thừa số nguyên tố

1. Khái niệm Ước số và Bội số

Định nghĩa

Cho hai số nguyên a và b ($b \neq 0$). Nếu có số nguyên k sao cho $a = b \times k$, ta nói:

- b là **ước số** của a (ký hiệu: $b \mid a$)
- a là **bội số** của b

Ví dụ

Xét số $N = 12$:

- Các ước số dương của 12 là: $\{1, 2, 3, 4, 6, 12\}$
- Bội số dương của 12 là: $\{12, 24, 36, 48, \dots\}$

Tính đối xứng của Ước số

Chứng minh tính đối xứng qua $\sqrt{N}$

Giả sử d là một ước của N. Khi đó sẽ tồn tại số nguyên k sao cho:

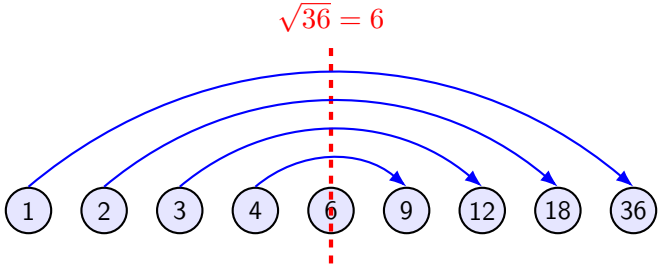
$$N = d \times k$$

Ta nhận thấy:

- Nếu $d \leq \sqrt{N}$ thì $k \geq \sqrt{N}$
- Nếu $d > \sqrt{N}$ thì $k < \sqrt{N}$

$\Longrightarrow$ Các ước số luôn xuất hiện theo **từng cặp** (d, k). Chắc chắn trong mỗi cặp sẽ có ít nhất một số $\leq \sqrt{N}$.

Minh họa trực quan tính đối xứng (Ví dụ: $N = 36$)



Trường hợp đặc biệt:
$d = N/d = 6$

Lưu ý

Khi N là số chính phương (như 36), cặp ước đối xứng của $\sqrt{N}$ chính là $(\sqrt{N}, \sqrt{N})$. Cần cẩn thận để **không đếm trùng** hoặc **cộng trùng** giá trị này!

Thuật toán đếm số lượng và tính tổng ước số

```cpp
long long count_divisors(long long N) {
    long long count = 0;
    for (long long i = 1; i * i <= N; i++) {
        if (N % i == 0) {
            count++; // Uoc i
            if (i * i != N) {
                count++; // Uoc doi xung N/i
            }
        }
    }
    return count;
}
```

Độ phức tạp

- **Thời gian:** $\mathcal{O}(\sqrt{N})$
- **Không gian:** $\mathcal{O}(1)$

Mẹo

Viết `i * i <= N` an toàn hơn `i <= sqrt(N)` vì tránh sai số dấu phẩy động. Tuy nhiên cẩn thận tràn số (overflow) nếu N lớn.

2. Số nguyên tố (Prime Numbers)

Khái niệm

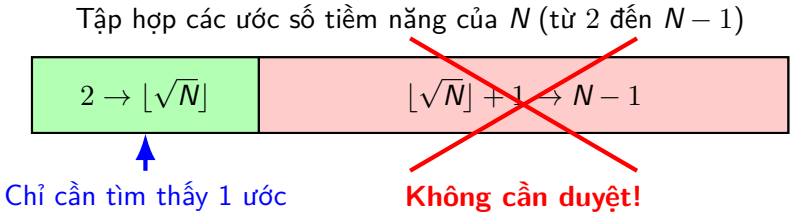
Số nguyên tố là số nguyên **lớn hơn 1**, chỉ có đúng 2 ước số dương là 1 và chính nó.

- Ví dụ: $2, 3, 5, 7, 11, 13, 17, \ldots$
- Số 2 là số nguyên tố chẵn duy nhất.
- Hợp số: Các số nguyên > 1 không phải là số nguyên tố (có > 2 ước).
- Số 0 và 1 **không phải** số nguyên tố, cũng không phải hợp số.

Ý tưởng Kiểm tra số nguyên tố

Dựa vào tính đối xứng của ước số, nếu N là hợp số thì N chắc chắn phải có ít nhất một ước d thỏa mãn $2 \leq d \leq \sqrt{N}$. Nếu kiểm tra trong đoạn này không có ước nào $\Longrightarrow$ N là số nguyên tố!

Minh họa Không gian tìm kiếm $O(\sqrt{N})$



Thuật toán kiểm tra số nguyên tố

```
bool isPrime(long long n) {
    if (n < 2) return false;
    for (long long i = 2; i * i <= n; i++) {
        if (n % i == 0)
            return false;
    }
    return true;
}
```

Độ phức tạp

- Thời gian: $\mathcal{O}(\sqrt{N})$.
- Không gian: $\mathcal{O}(1)$.

Kinh nghiệm thi đấu

Thuật toán trên có thể kiểm tra tính nguyên tố của $N \leq 10^{14}$ trong giới hạn thời gian cho phép.

3. Khái niệm GCD và LCM

Ước chung lớn nhất (GCD)

Greatest Common Divisor
Là số nguyên dương lớn nhất chia hết cả a và b.
Ký hiệu: $\gcd(a, b)$

Ví dụ: $\gcd(12, 18) = 6$

Bội chung nhỏ nhất (LCM)

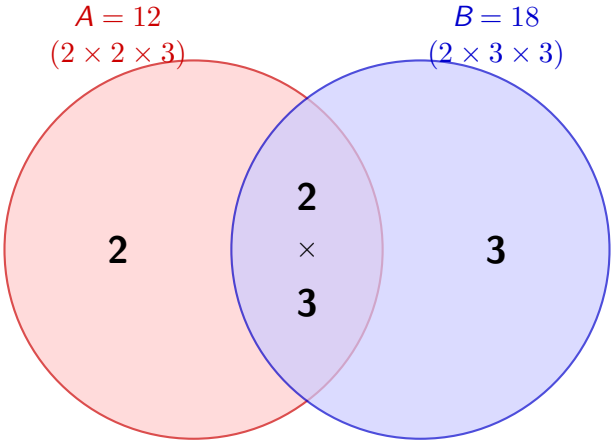
Least Common Multiple
Là số nguyên dương nhỏ nhất chia hết cho cả a và b.
Ký hiệu: $\text{lcm}(a, b)$

Ví dụ: $\text{lcm}(12, 18) = 36$

Lưu ý

$\gcd(a, 0) = a$. Mọi số nguyên đều là ước của 0 (trừ 0).

Minh họa trực quan GCD / LCM qua tập hợp thừa số nguyên tố



Phần giao (GCD) $= 2 \times 3 = 6$

Phần hợp (LCM) $= 2 \times (2 \times 3) \times 3 = 36$

Thuật toán Euclid tính GCD

Ý tưởng Thuật toán Euclid

Dựa trên tính chất toán học: $\gcd(a, b) = \gcd(b, a \pmod b)$.
Quá trình đệ quy kết thúc khi $b = 0$, lúc đó $\gcd(a, 0) = a$.

C++ Code (Đệ quy)

```cpp
long long gcd(long long a,long long b){
    if (b == 0) return a;
    return gcd(b, a % b);
}
```

C++ Code (Không đệ quy)

```cpp
long long gcd(long long a, long long b)
    {
    while (b != 0) {
        long long r = a % b;
        a = b;
        b = r;
    }
    return a;
}
```

Mối liên hệ giữa GCD và LCM

Chứng minh Mối quan hệ tích

Từ biểu đồ Venn phía trên, ta dễ dàng nhận thấy: Tích của $\gcd(a, b)$ (phần giao) và $\text{lcm}(a, b)$ (phần hợp) chính là tích của toàn bộ tập thừa số của A nhân với toàn bộ tập thừa số của B.

$$a \times b = \gcd(a, b) \times \text{lcm}(a, b)$$

$$\Longrightarrow \text{lcm}(a, b) = \frac{a \times b}{\gcd(a, b)}$$

Tránh tràn số

Khi cài đặt tính LCM, **KHÔNG** nên viết `a * b / gcd(a, b)` vì `a * b` có thể gây tràn kiểu `long long`. Hãy viết: `(a / gcd(a, b)) * b`. Vì `a` luôn chia hết cho `gcd(a, b)`, phép chia sẽ luôn nguyên và triệt tiêu rủi ro tràn số sớm!

4. Định lý cơ bản của Số học

Định lý

Mọi số nguyên dương $N > 1$ đều có thể được biểu diễn một cách **duy nhất** dưới dạng tích của các số nguyên tố (nếu bỏ qua thứ tự các thừa số).

Dạng phân tích chuẩn (Prime Factorization):

$$N = p_1^{e_1} \times p_2^{e_2} \times p_3^{e_3} \times \cdots \times p_k^{e_k}$$

Trong đó:

- $p_1 < p_2 < \cdots < p_k$ là các **thừa số nguyên tố**.
- $e_1, e_2, \ldots, e_k$ là **số mũ** (số lần xuất hiện của thừa số nguyên tố tương ứng, $e_i \geq 1$).

Ví dụ

$60 = 2 \times 2 \times 3 \times 5 = 2^2 \times 3^1 \times 5^1$

Thuật toán phân tích thừa số nguyên tố $\mathcal{O}(\sqrt{N})$

Ý tưởng

1. Duyệt i từ 2 đến $\sqrt{N}$.
2. Nếu i là một ước của N (tức là $N \pmod{i} == 0$), ta chia N cho i liên tục cho đến khi N không còn chia hết cho i nữa. Mỗi lần chia, tăng số mũ e_i lên 1.
3. Vì ta duyệt từ bé đến lớn và đã chia hết cho các ước nhỏ, khi gặp một số i chia hết N, **chắc chắn i là số nguyên tố**.

Điều kiện đặc biệt ở cuối thuật toán

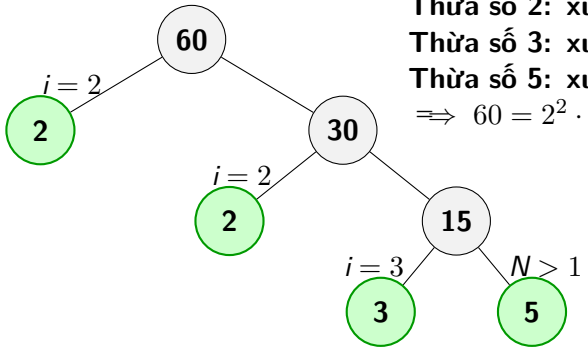
Sau khi vòng lặp kết thúc, nếu N còn lại **lớn hơn 1**, thì phần còn lại này chính là một **số nguyên tố duy nhất có giá trị** $> \sqrt{N}$. (Ví dụ: $N = 14$, duyệt tới $\sqrt{14} = 3$, chia 2 còn $N = 7$. Cuối cùng $7 > 1$, ta nạp 7 vào kết quả).

Code phân tích thừa số nguyên tố

Phân tích N thành các cặp (Số nguyên tố, Số mũ)

```cpp
vector<pair<long long, int>> primeFactorize(long long n) {
    vector<pair<long long, int>> factors;
    for (long long i = 2; i * i <= n; i++) {
        if (n % i == 0) {
            int count = 0;
            while (n % i == 0) {
                count++;
                n /= i;
            }
            factors.push_back({i, count});
        }
    }
    if (n > 1) {
        factors.push_back({n, 1});
    }
    return factors;
}
```

Minh họa quá trình phân tích $N = 60$



Kết quả:
Thừa số 2: xuất hiện 2 lần
Thừa số 3: xuất hiện 1 lần
Thừa số 5: xuất hiện 1 lần
$\Longrightarrow 60 = 2^2 \cdot 3^1 \cdot 5^1$

Mối liên hệ Toán học quan trọng

Cho $N = p_1^{e_1} \times p_2^{e_2} \times \cdots \times p_k^{e_k}$.

Công thức số lượng ước

- **Số lượng ước số của N:**

$$d(N) = (e_1 + 1) \times (e_2 + 1) \times \cdots \times (e_k + 1)$$

(Mỗi thừa số p_i có thể chọn số mũ từ 0 đến e_i $\Longrightarrow$ có $e_i + 1$ cách chọn)

Mối liên hệ giữa GCD, LCM và Số mũ nguyên tố

Mối liên hệ

Cho 2 số A và B được phân tích thừa số nguyên tố (bổ sung các số mũ 0 để hai số có cùng tập thừa số nguyên tố):

$A = p_1^{a_1} \times p_2^{a_2} \times \cdots \times p_k^{a_k}$

$B = p_1^{b_1} \times p_2^{b_2} \times \cdots \times p_k^{b_k}$

Khi đó, GCD và LCM được xác định bằng cách so sánh các số mũ:

- $\gcd(A, B) = p_1^{\min(a_1,b_1)} \times p_2^{\min(a_2,b_2)} \times \cdots \times p_k^{\min(a_k,b_k)}$
- $\text{lcm}(A, B) = p_1^{\max(a_1,b_1)} \times p_2^{\max(a_2,b_2)} \times \cdots \times p_k^{\max(a_k,b_k)}$

Mối liên hệ giữa GCD, LCM và Số mũ nguyên tố

Ví dụ minh họa: $A = 60$ và $B = 72$

Phân tích: $A = 60 = 2^2 \times 3^1 \times 5^1$ và $B = 72 = 2^3 \times 3^2 \times 5^0$.

Thành phần	$\mathbf{p = 2}$	$\mathbf{p = 3}$	$\mathbf{p = 5}$	**Kết quả**
Số mũ của A (a_i)	2	1	1	60
Số mũ của B (b_i)	3	2	0	72
$\gcd(A, B)$ *(chọn min)*	$2^{\mathbf{2}}$	$3^{\mathbf{1}}$	$5^{\mathbf{0}}$	$\Longrightarrow 4 \times 3 \times 1 = \mathbf{12}$
$\mathrm{lcm}(A, B)$ *(chọn max)*	$2^{\mathbf{3}}$	$3^{\mathbf{2}}$	$5^{\mathbf{1}}$	$\Longrightarrow 8 \times 9 \times 5 = \mathbf{360}$

Thank You!